

Student ID Card Request Module

Laravel 13 · Request & Approval Workflow · Students Domain

This module lets students request ID card printing from the portal (new / lost / damaged / renewal), and gives admins a review queue to approve, reject, print, and issue cards, with barcode-enabled printable cards and full activity logging.

1. Module Structure

```
App\Models\Students\StudentIdCardRequest
App\Enums\Students\IdCardRequestStatusEnum
App\Enums\Students\IdCardRequestReasonEnum    (new / lost / damaged / renewal)
App\Http\Controllers\Students\IdCardRequestController    (student portal)
App\Http\Controllers\Admin\Students\IdCardRequestController    (admin/registry)
App\Services\Students\StudentIdCardService              (PDF/barcode)
App\Policies\Students\StudentIdCardRequestPolicy
App\Notifications\Students\IdCardRequestStatusUpdated
```

2. Migration

database/migrations/xxxx_xx_xx_create_student_id_card_requests_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('student_id_card_requests', function (Blueprint $table) {
            $table->id();
            $table->foreignId('tenant_id')->constrained()->cascadeOnDelete();
            $table->foreignId('student_id')->constrained('students')->cascadeOnDelete();

            $table->string('status')->default('pending'); // pending, approved, rejected, printed, issued
            $table->string('reason'); // new, lost, damaged, renewal
            $table->text('notes')->nullable(); // student's comment, e.g. "lost at gym"

            $table->foreignId('reviewed_by')->nullable()->constrained('users')->nullOnDelete();
            $table->timestamp('reviewed_at')->nullable();
            $table->text('rejection_reason')->nullable();

            $table->foreignId('printed_by')->nullable()->constrained('users')->nullOnDelete();
            $table->timestamp('printed_at')->nullable();

            $table->foreignId('issued_by')->nullable()->constrained('users')->nullOnDelete();
            $table->timestamp('issued_at')->nullable();

            $table->string('serial_number')->nullable()->unique(); // printed card's barcode payload

            $table->timestamps();
            $table->softDeletes();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('student_id_card_requests');
    }
};
```

3. Enums

App/Enums/Students/IdCardRequestStatusEnum.php

```
<?php

namespace App\Enums\Students;

enum IdCardRequestStatusEnum: string
{
    case PENDING = 'pending';
    case APPROVED = 'approved';
    case REJECTED = 'rejected';
    case PRINTED = 'printed';
    case ISSUED = 'issued';

    public function label(): string
    {
        return match ($this) {
            self::PENDING => 'Pending Review',
            self::APPROVED => 'Approved',
            self::REJECTED => 'Rejected',
            self::PRINTED => 'Printed',
            self::ISSUED => 'Issued to Student',
        };
    }

    public function color(): string
    {
        return match ($this) {
            self::PENDING => 'warning',
            self::APPROVED => 'info',
            self::REJECTED => 'danger',
            self::PRINTED => 'primary',
            self::ISSUED => 'success',
        };
    }
}
```

App/Enums/Students/IdCardRequestReasonEnum.php

```
<?php

namespace App\Enums\Students;

enum IdCardRequestReasonEnum: string
{
    case NEW = 'new';
    case LOST = 'lost';
    case DAMAGED = 'damaged';
    case RENEWAL = 'renewal';

    public function label(): string
    {
        return match ($this) {
            self::NEW => 'First Time / New Student',
            self::LOST => 'Lost Card',
            self::DAMAGED => 'Damaged Card',
            self::RENEWAL => 'Renewal / Expired',
        };
    }
}
```

4. Model

App/Models/Students/StudentIdCardRequest.php

```
<?php

namespace App\Models\Students;

use App\Enums\Students\IdCardRequestReasonEnum;
use App\Enums\Students\IdCardRequestStatusEnum;
use App\Models\Users\User;
use App\Traits\BelongsToTenant;
use App\Traits\Filterable;
use App\Traits\Paginatable;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\SoftDeletes;
use Spatie\Activitylog\LogOptions;
use Spatie\Activitylog\Traits\LogsActivity;

class StudentIdCardRequest extends Model
{
    use BelongsToTenant, Filterable, HasFactory, LogsActivity, Paginatable, SoftDeletes;

    protected $fillable = [
        'tenant_id',
        'student_id',
        'status',
        'reason',
        'notes',
        'reviewed_by',
        'reviewed_at',
        'rejection_reason',
        'printed_by',
        'printed_at',
        'issued_by',
        'issued_at',
        'serial_number',
    ];

    protected function casts(): array
    {
        return [

```

```

        'status' => IdCardRequestStatusEnum::class,
        'reason' => IdCardRequestReasonEnum::class,
        'reviewed_at' => 'datetime',
        'printed_at' => 'datetime',
        'issued_at' => 'datetime',
    ];
}

public function student(): BelongsTo
{
    return $this->belongsTo(Student::class);
}

public function reviewer(): BelongsTo
{
    return $this->belongsTo(User::class, 'reviewed_by');
}

public function printer(): BelongsTo
{
    return $this->belongsTo(User::class, 'printed_by');
}

public function issuer(): BelongsTo
{
    return $this->belongsTo(User::class, 'issued_by');
}

public function approve(User $admin): void
{
    $this->update([
        'status' => IdCardRequestStatusEnum::APPROVED,
        'reviewed_by' => $admin->id,
        'reviewed_at' => now(),
    ]);
}

public function reject(User $admin, string $reason): void
{
    $this->update([
        'status' => IdCardRequestStatusEnum::REJECTED,
        'reviewed_by' => $admin->id,
        'reviewed_at' => now(),
        'rejection_reason' => $reason,
    ]);
}

public function markPrinted(User $admin, string $serialNumber): void
{
    $this->update([
        'status' => IdCardRequestStatusEnum::PRINTED,
        'printed_by' => $admin->id,
        'printed_at' => now(),
        'serial_number' => $serialNumber,
    ]);
}

public function markIssued(User $admin): void
{
    $this->update([
        'status' => IdCardRequestStatusEnum::ISSUED,
        'issued_by' => $admin->id,
        'issued_at' => now(),
    ]);
}

public function getActivitylogOptions(): LogOptions
{
    return LogOptions::defaults()
        ->logFillable()
        ->useLogName('StudentIdCardRequest')
        ->logOnlyDirty()
        ->dontSubmitEmptyLogs();
}
}

```

Add to the existing **Student** model:

```

public function idCardRequests(): HasMany
{
    return $this->hasMany(StudentIdCardRequest::class, 'student_id');
}

public function latestIdCardRequest(): HasOne
{
    return $this->hasOne(StudentIdCardRequest::class, 'student_id')->latestOfMany();
}

```

5. Policy

`App/Policies/Students/StudentIdCardRequestPolicy.php`

```
<?php

namespace App\Policies\Students;

use App\Models\Students\StudentIdCardRequest;
use App\Models\Users\User;

class StudentIdCardRequestPolicy
{
    public function view(User $user, StudentIdCardRequest $request): bool
    {
        return $user->student?->id === $request->student_id
            || $user->can('students.id-cards.manage');
    }

    public function create(User $user): bool
    {
        return (bool) $user->student;
    }

    public function review(User $user): bool
    {
        return $user->can('students.id-cards.manage');
    }
}
```

Adjust this to fit your existing Gate/Policy RBAC pattern (Role/Permission enums + central Gate registry + `can()` dependency equivalent already used across your FastAPI projects).

6. Student-Facing Controller

App/Http/Controllers/Students/IdCardRequestController.php

```
<?php

namespace App\Http\Controllers\Students;

use App\Enums\Students\IdCardRequestStatusEnum;
use App\Http\Controllers\Controller;
use App\Models\Students\StudentIdCardRequest;
use Illuminate\Http\Request;

class IdCardRequestController extends Controller
{
    public function index(Request $request)
    {
        $requests = $request->user()->student
            ->idCardRequests()
            ->latest()
            ->paginate();

        return view('students.id-card-requests.index', compact('requests'));
    }

    public function store(Request $request)
    {
        $student = $request->user()->student;

        // Prevent duplicate open requests
        abort_if(
            $student->idCardRequests()
                ->whereIn('status', [IdCardRequestStatusEnum::PENDING, IdCardRequestStatusEnum::APPROVED])
                ->exists(),
            422,
            'You already have an active ID card request.'
        );

        $data = $request->validate([
            'reason' => 'required|in:new,lost,damaged,renewal',
            'notes' => 'nullable|string|max:500',
        ]);

        $student->idCardRequests()->create([
            'tenant_id' => $student->tenant_id,
            'status' => IdCardRequestStatusEnum::PENDING,
            ...$data,
        ]);

        return back()->with('success', 'ID card request submitted.');
```

7. Admin Controller

App/Http/Controllers/Admin/Students/IdCardRequestController.php

```
<?php

namespace App\Http\Controllers\Admin\Students;

use App\Enums\Students\IdCardRequestStatusEnum;
use App\Http\Controllers\Controller;
use App\Models\Students\StudentIdCardRequest;
use App\Services\Students\StudentIdCardService;
use Illuminate\Http\Request;

class IdCardRequestController extends Controller
{
    public function __construct(private StudentIdCardService $idCardService)
    {
        $this->authorizeResource(StudentIdCardRequest::class, 'idCardRequest');
    }

    public function index(Request $request)
    {
        $requests = StudentIdCardRequest::with(['student.user'])
            ->when($request->status, fn ($q, $status) => $q->where('status', $status))
            ->latest()
            ->paginate(25);

        return view('admin.students.id-card-requests.index', compact('requests'));
    }

    public function approve(StudentIdCardRequest $idCardRequest)
    {
        $this->authorize('review', StudentIdCardRequest::class);

        $idCardRequest->approve(auth()->user());

        return back()->with('success', 'Request approved.');
```

```

    }

    public function reject(Request $request, StudentIdCardRequest $idCardRequest)
    {
        $this->authorize('review', StudentIdCardRequest::class);

        $data = $request->validate(['rejection_reason' => 'required|string|max:500']);

        $idCardRequest->reject(auth()->user(), $data['rejection_reason']);

        return back()->with('success', 'Request rejected.');
```

```

    }

    public function print(StudentIdCardRequest $idCardRequest)
    {
        $this->authorize('review', StudentIdCardRequest::class);

        abort_unless(
            $idCardRequest->status === IdCardRequestStatusEnum::APPROVED,
            422,
            'Only approved requests can be printed.'
        );

        $serial = strtoupper('ID-' . $idCardRequest->student->student_number . '-' . now()->format('ym'));

        $idCardRequest->markPrinted(auth()->user(), $serial);

        return $this->idCardService
            ->renderSingle($idCardRequest->student, $serial)
            ->stream("student-id-{$idCardRequest->student->student_number}.pdf");
    }

    public function issue(StudentIdCardRequest $idCardRequest)
    {
        $this->authorize('review', StudentIdCardRequest::class);

        $idCardRequest->markIssued(auth()->user());

        return back()->with('success', 'Card marked as issued to student.');
```

```

    }
}

```

8. Routes

routes/web.php

```
Route::middleware(['auth', 'student'])->prefix('portal/id-card')->name('students.id-card-requests.')->group(function () {
    Route::get('/', [IdCardRequestController::class, 'index'])->name('index');
    Route::post('/', [IdCardRequestController::class, 'store'])->name('store');
});

Route::middleware(['auth', 'can:students.id-cards.manage'])->prefix('admin/students/id-card-requests')->name('admin.students.id-card-requests.')->group(function () {
    Route::get('/', [Admin\Students\IdCardRequestController::class, 'index'])->name('index');
    Route::post('/{idCardRequest}/approve', [Admin\Students\IdCardRequestController::class, 'approve'])->name('approve');
    Route::post('/{idCardRequest}/reject', [Admin\Students\IdCardRequestController::class, 'reject'])->name('reject');
    Route::get('/{idCardRequest}/print', [Admin\Students\IdCardRequestController::class, 'print'])->name('print');
    Route::post('/{idCardRequest}/issue', [Admin\Students\IdCardRequestController::class, 'issue'])->name('issue');
});
```

9. Workflow Summary

Student submits request → PENDING Admin reviews → APPROVED / REJECTED Admin prints card → PRINTED (serial number generated, embedded in barcode) Admin hands card over → ISSUED

10. Open Decisions

Question	Why it matters
One active card per student, or full history?	The unique-serial + status-per-request design above already supports reissue history for lost/damaged cards — usually what registries want.
Student notifications on status change?	Can wire up <code>IdCardRequestStatusUpdated</code> using your existing notification setup.
Bulk approval / print?	For new-intake cohorts, admins often want to select many pending requests and approve + print in one batch — changes the controller/UI shape.